

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

QUESTIONS:

Total Marks: 25

1: XML Parsing Algorithm

Write a detailed **pseudocode algorithm** to parse an XML document that contains structured data such as user or product details. Your algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

2: MVC Workflow

Write a **pseudocode representation** of the MVC (Model–View–Controller) workflow in a web application. Your answer should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View generates and returns the response to the user. Include all major steps involved in request handling and response generation.

3: JSP Request Flow

Write a detailed **pseudocode algorithm** to explain the request–response flow in a JSP-based web application. Your algorithm should include steps such as client request initiation, server processing, interaction with backend components (like Servlets or database), and generation of dynamic content that is sent back to the client.

4: Database Retrieval

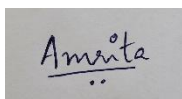
Write a **pseudocode algorithm** to retrieve data from a database. Your answer should include steps for establishing a database connection, executing SQL queries, processing the result set, and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

5: PHP Validation

Write a **pseudocode algorithm** for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats (such as email and password), handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

Code of Conduct:

I AMRITA ANAIKUMAR – 192471035 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

CO4 – AT2

ANSWERS

1. XML Parsing Algorithm

2.

Aim

To parse an XML document containing structured user/product details, traverse its hierarchical nodes, extract required information, and display it in a readable format.

Pseudocode

ALGORITHM XML_Parsing

BEGIN

STEP 1: Start

STEP 2: Load the XML document

XMLDocument ← LOAD_XML("data.xml")

STEP 3: Check whether XML document is loaded successfully

IF XMLDocument = NULL THEN

DISPLAY "Error: Unable to load XML file"

STOP

END IF

STEP 4: Access the root element

Root ← GET_ROOT(XMLDocument)

STEP 5: Display root element

DISPLAY "Root Element: ", Root.Name

STEP 6: Access the required child elements

Records ← GET_CHILDREN(Root)

STEP 7: Traverse through each record

FOR EACH Record IN Records DO

DISPLAY "-----"

STEP 8: Access the elements inside the record

ID ← GET_VALUE(Record, "ID")

Name ← GET_VALUE(Record, "Name")

Email ← GET_VALUE(Record, "Email")

STEP 9: Display extracted information

DISPLAY "ID: ", ID

DISPLAY "Name: ", Name

DISPLAY "Email: ", Email

STEP 10: Handle hierarchical child nodes

Details ← GET_CHILDREN(Record)

FOR EACH Detail IN Details DO

DISPLAY Detail.Name, ": ", Detail.Value

END FOR

END FOR

STEP 11: Display "XML parsing completed"

END

Example XML

```
<users>
  <user>
    <ID>101</ID>
    <Name>John</Name>
    <Email>john@gmail.com</Email>
  </user>
  <user>
    <ID>102</ID>
    <Name>Mary</Name>
    <Email>mary@gmail.com</Email>
  </user>
</users>
```

Sample Output

Root Element: users

ID: 101

Name: John

Email: john@gmail.com

ID: 102

Name: Mary

Email: mary@gmail.com

XML parsing completed

Explanation

The algorithm first loads the XML document and checks for errors. It then accesses the root node and traverses its child nodes using a loop. Required values are extracted from each node and displayed. Nested nodes are also traversed, allowing the algorithm to correctly handle the hierarchical structure of XML.

2. MVC Workflow

The question requires the workflow to show the user request, Controller, Model, database interaction, View and final response.

Aim

To represent the complete MVC workflow of a web application from receiving a user request to returning the response.

Pseudocode

ALGORITHM MVC_Workflow

BEGIN

STEP 1: Start

STEP 2: User sends a request

Request ← RECEIVE_REQUEST()

STEP 3: Send request to Controller

Controller ← GET_CONTROLLER(Request)

STEP 4: Controller receives and analyzes request

Action ← IDENTIFY_ACTION(Request)

STEP 5: Controller calls the Model

Data ← MODEL_PROCESS(Action)

STEP 6: Model connects to the database

Connection ← CONNECT_TO_DATABASE()

STEP 7: Check database connection

IF Connection = FAILED THEN

 DISPLAY "Database connection error"

 STOP

END IF

STEP 8: Execute required database operation

Result ← EXECUTE_QUERY(Connection, Action)

STEP 9: Process database result

Data ← PROCESS_RESULT(Result)

STEP 10: Close database connection

CLOSE(Connection)

STEP 11: Controller receives data from Model

ResponseData ← Data

STEP 12: Controller sends data to View

View ← SELECT_VIEW(Action)

STEP 13: View generates the response

Response ← GENERATE_VIEW(View, ResponseData)

STEP 14: Send response to user

SEND_RESPONSE(Response)

STEP 15: End

END

MVC Flow Diagram

User

|

| Request

v

Controller

|

| Request data

v

Model

|

| SQL Query

v

Database

|

| Result

v

Model

|

| Data

v

Controller

|

| Data

v

View

|

| HTML Response

v

User

Explanation

The Controller receives the user's request and determines the required operation. It communicates with the Model, which performs business logic and interacts with the database. The Model returns the required data to the Controller. The Controller passes the data to the **View**, which generates the final response and sends it back to the user.

3. JSP Request Flow

The question specifically requires the JSP flow to include client request, server processing, Servlets/backend/database interaction and dynamic content generation.

Aim

To explain the request-response process of a JSP-based web application.

Pseudocode

ALGORITHM JSP_Request_Flow

BEGIN

STEP 1: Start

STEP 2: Client sends a request

Request $\leftarrow$ CLIENT_REQUEST(URL)

STEP 3: Web server receives the request

STEP 4: Identify the requested JSP page

JSPPage $\leftarrow$ FIND_JSP(Request)

STEP 5: Check whether JSP page is already compiled

IF JSPPage NOT_COMPILED THEN

TRANSLATE_JSP_TO_SERVLET(JSPPage)

COMPILE_SERVLET()

END IF

STEP 6: Execute the generated Servlet

STEP 7: Servlet processes the request

RequestData $\leftarrow$ GET_REQUEST_DATA(Request)

STEP 8: If backend data is required

Connection $\leftarrow$ CONNECT_TO_DATABASE()

STEP 9: Execute required database operation

Result $\leftarrow$ EXECUTE_QUERY(Connection, RequestData)

STEP 10: Process the result

Data $\leftarrow$ PROCESS_RESULT(Result)

STEP 11: Close database connection

CLOSE(Connection)

STEP 12: Send data to JSP

STEP 13: JSP generates dynamic HTML content

HTML $\leftarrow$ GENERATE_DYNAMIC_CONTENT(Data)

STEP 14: Server sends HTML response

SEND_RESPONSE_TO_CLIENT(HTML)

STEP 15: Browser displays the response

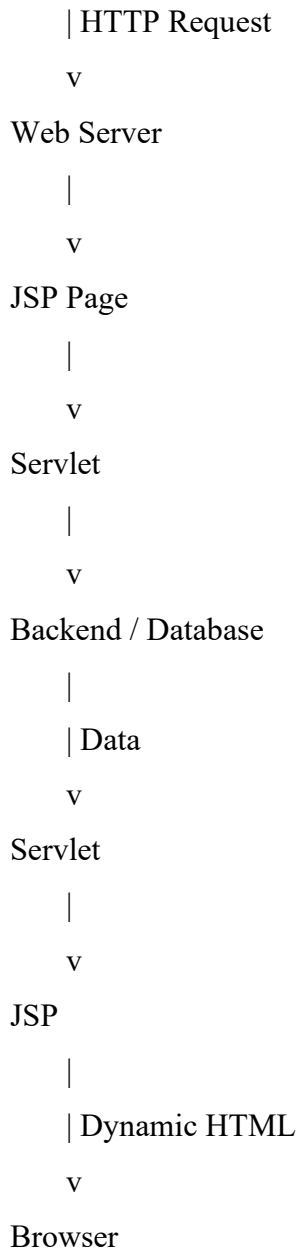
STEP 16: End

END

JSP Request Flow Diagram

Client / Browser

|



Sample Output

Student Details

Student ID : 101

Name : John

Course : B.Tech

Explanation

When the client requests a JSP page, the web server processes the request. The JSP page is translated into a Servlet if required. The Servlet processes the request and can communicate with backend components such as a database. The retrieved data is passed to the JSP, which generates dynamic HTML content and sends it back to the browser.

4. Database Retrieval

The question requires the algorithm to show database connection, SQL execution, result-set processing and displaying the retrieved data.

Aim

To retrieve data from a database by establishing a connection, executing an SQL query, processing the result set and displaying the retrieved records.

Pseudocode

ALGORITHM Database_Retrieval

BEGIN

STEP 1: Start

STEP 2: Define database details

URL ← "Database URL"

USERNAME ← "Database Username"

PASSWORD ← "Database Password"

STEP 3: Establish database connection

Connection ← CONNECT_DATABASE(URL, USERNAME, PASSWORD)

STEP 4: Check connection

IF Connection = FAILED THEN

DISPLAY "Database connection failed"

STOP

END IF

STEP 5: Create SQL query

Query ← "SELECT * FROM Student"

STEP 6: Execute SQL query

ResultSet ← EXECUTE_QUERY(Connection, Query)

STEP 7: Check whether records exist

IF ResultSet IS EMPTY THEN

DISPLAY "No records found"

ELSE

STEP 8: Process each record

WHILE HAS_NEXT(ResultSet) DO

Record ← NEXT(ResultSet)

```

        ID ← GET_VALUE(Record, "ID")
        Name ← GET_VALUE(Record, "Name")
        Course ← GET_VALUE(Record, "Course")
        STEP 9: Display record
        DISPLAY "ID: ", ID
        DISPLAY "Name: ", Name
        DISPLAY "Course: ", Course
        DISPLAY "-----"
    END WHILE
END IF
STEP 10: Close result set
    CLOSE(ResultSet)
STEP 11: Close database connection
    CLOSE(Connection)
STEP 12: Display completion message
    DISPLAY "Data retrieval completed"
STEP 13: End
END

```

Sample Database

Student

```

-----
ID   Name   Course
-----
101  John    BCA
102  Mary    B.Tech
103  David    MCA

```

Sample Output

```

ID: 101
Name: John
Course: BCA
-----

```

ID: 102

Name: Mary

Course: B.Tech

ID: 103

Name: David

Course: MCA

Data retrieval completed

Database Flow

Start

|

v

Connect to Database

|

v

Execute SQL Query

|

v

Get Result Set

|

v

Are Records Available?

|

No

|

Yes

|

v

|

v

Display

Process Each

"No Records" Record

|

v

Display Data

|

v

Close ResultSet

|

v

Close Connection

|

v

End

5. PHP Validation

The question requires validation for empty fields, email format, password format, error messages and submission only when all validations pass.

Aim

To validate user input submitted through a PHP web form and accept the form only when all input values are valid.

Pseudocode

ALGORITHM PHP_Form_Validation

BEGIN

STEP 1: Start

STEP 2: Initialize error messages

Errors ← EMPTY

STEP 3: Receive form data

Name ← GET_FORM_VALUE("name")

Email ← GET_FORM_VALUE("email")

Password ← GET_FORM_VALUE("password")

STEP 4: Remove unnecessary spaces

Name ← TRIM(Name)

Email ← TRIM>Email)

Password ← TRIM>Password)

STEP 5: Validate Name

```
IF Name = EMPTY THEN
    ADD "Name is required" TO Errors
END IF
```

STEP 6: Validate Email

```
IF Email = EMPTY THEN
    ADD "Email is required" TO Errors
ELSE IF EMAIL_FORMAT_IS_INVALID(Email) THEN
    ADD "Invalid email format" TO Errors
END IF
```

STEP 7: Validate Password

```
IF Password = EMPTY THEN
    ADD "Password is required" TO Errors
ELSE IF LENGTH(Password) < 8 THEN
    ADD "Password must contain at least 8 characters" TO Errors
END IF
```

STEP 8: Check validation result

```
IF Errors IS NOT EMPTY THEN
    DISPLAY "Form contains errors"
    FOR EACH Error IN Errors DO
        DISPLAY Error
    END FOR
    DISPLAY "Please correct the errors"
```

ELSE

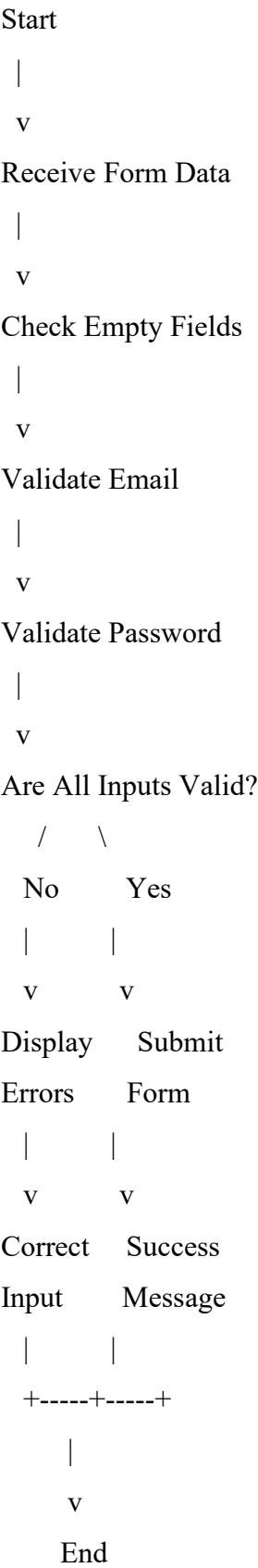
```
    STEP 9: All inputs are valid
    DISPLAY "Validation successful"
    STEP 10: Process form submission
    SUBMIT_FORM(Name, Email, Password)
    DISPLAY "Form submitted successfully"
```

END IF

STEP 11: End

END

Validation Flow Diagram



Sample Input 1

Name : John

Email : john@gmail.com

Password : John@1234

Sample Output 1

Validation successful

Form submitted successfully

Sample Input 2

Name :

Email : john

Password : 123

Sample Output 2

Form contains errors

Name is required

Invalid email format

Password must contain at least 8 characters

Please correct the errors